

ICPC Team Codebook

Team Template Season 2026

3
CATEGORIES

3
TEMPLATES

74
SOURCE LINES

A4 - two-column - monochrome - Consolas

Quick Index

1	Data Structures	2
1.1	Disjoint Set Union	2
2	Graph	2
2.1	Dijkstra Shortest Paths	2
3	Math	2
3.1	Modular Power and Inverse	2

1 Data Structures

1.1 Disjoint Set Union

FILE 01-data-structures/disjoint_set_union.cpp

LINES 28

USE Maintain components with path compression and union by size.

COST $O(\alpha(n))$ amortized per operation

TAGS dsu, union-find, connectivity

```

1 struct DSU {
2     std::vector<int> parent, size;

4     explicit DSU(int n) : parent(n), size(n, 1) {
5         std::iota(parent.begin(), parent.end(), 0);
6     }

8     int find(int x) {
9         while (x != parent[x]) {
10             parent[x] = parent[parent[x]];
11             x = parent[x];
12         }
13         return x;
14     }

16     bool unite(int a, int b) {
17         a = find(a);
18         b = find(b);
19         if (a == b) return false;
20         if (size[a] < size[b]) std::swap(a, b);
21         parent[b] = a;
22         size[a] += size[b];
23         return true;
24     }

26     bool same(int a, int b) { return find(a) == find(b); }
27     int component_size(int x) { return size[find(x)]; }
28 };

```

2 Graph

2.1 Dijkstra Shortest Paths

FILE 02-graph/dijkstra.cpp

LINES 30

USE Single-source shortest paths in a graph with non-negative edge weights.

COST $O((n + m) \log n)$

TAGS graph, shortest-path, priority-queue

```

1 using i64 = long long;
2 constexpr i64 INF = std::numeric_limits<i64>::max() / 4;

4 struct Edge {
5     int to;
6     i64 weight;
7 };

9 std::vector<i64> dijkstra(const std::vector<std::vector<Edge>>& graph, int source) {
10     const int n = static_cast<int>(graph.size());
11     std::vector<i64> distance(n, INF);
12     using State = std::pair<i64, int>;
13     std::priority_queue<State, std::vector<State>, std::greater<State>> queue;

```

```

15     distance[source] = 0;
16     queue.emplace(0, source);
17     while (!queue.empty()) {
18         auto [current_distance, vertex] = queue.top();
19         queue.pop();
20         if (current_distance != distance[vertex]) continue;

22         for (const Edge& edge : graph[vertex]) {
23             const i64 candidate = current_distance + edge.weight;
24             if (candidate >= distance[edge.to]) continue;
25             distance[edge.to] = candidate;
26             queue.emplace(candidate, edge.to);
27         }
28     }
29     return distance;
30 }

```

3 Math

3.1 Modular Power and Inverse

FILE 03-math/modular_power.cpp

LINES 16

USE Binary exponentiation and multiplicative inverse modulo a prime.

COST $O(\log \text{exponent})$

TAGS modular-arithmetic, number-theory, fast-power

```

1 using i64 = long long;

3 i64 modular_power(i64 base, i64 exponent, i64 modulus) {
4     base %= modulus;
5     i64 result = 1 % modulus;
6     while (exponent > 0) {
7         if (exponent & 1) result = static_cast<__int128>(result) * base % modulus;
8         base = static_cast<__int128>(base) * base % modulus;
9         exponent >>= 1;
10    }
11    return result;
12 }

14 i64 modular_inverse_prime(i64 value, i64 prime_modulus) {
15     return modular_power(value, prime_modulus - 2, prime_modulus);
16 }

```